

```

% Define system matrices from the paper
A = [0.72 -0.12; -0.38 0.38];
S = [0.25 -0.13; 0.15 0.15];
R = [0.58 -0.2; 0 0.49];

% Initialize LMI System
setlmis([]);

% Sector bounds and convergence rate
Ko=-1;
Kq=1;
alpha=0.01;
exp_term=exp(2*alpha);
I=eye(2); %identity matrix, 2x2
% Define the Variable P (The Lyapunov Matrix)
% P must be symmetric (Type 1) and positive definite
P = lmivar(1, [2, 1]);
J = lmivar(1, [2, 0]);
E = lmivar(1, [2,0]);

% q11:  $Kq^2 A'JA - 2KqKo A'EA - P$ 
lmiterm([-1, 1, 1, J], Kq^2 * A', A);
lmiterm([-1, 1, 1, E], -2 * Kq * Ko * A', A);
lmiterm([-1, 1, 1, P], -1, 1);

% q12:  $(Ko + Kq) A'E$ 
lmiterm([-1, 1, 2, E], (Ko + Kq) * A', 1);

% q13:  $Kq A'JR - 2KqKo A'ER - 0.5 A'$ 
lmiterm([-1, 1, 3, J], Kq * A', R);
lmiterm([-1, 1, 3, E], -2 * Kq * Ko * A', R);
lmiterm([-1, 1, 3, 0], -0.5 * A');

% q22:  $e^{(2*\alpha)}P - J - 2E$ 
lmiterm([-1, 2, 2, P], exp_term, 1);
lmiterm([-1, 2, 2, J], -1, 1);
lmiterm([-1, 2, 2, E], -2, 1);

% q23:  $e^{(2*\alpha)}PS + (Ko + Kq)ER$ 
lmiterm([-1, 2, 3, P], exp_term, S);
lmiterm([-1, 2, 3, E], (Ko + Kq), R);

% q33:  $e^{(2*\alpha)}S'PS + R'JR - 2KqKo R'ER - (R + R')/2$ 
% NOTE: For S'PS, the multiplier on the left must be (exp_term * S')
lmiterm([-1, 3, 3, P], exp_term * S', S);
lmiterm([-1, 3, 3, J], R', R);
lmiterm([-1, 3, 3, E], -2 * Kq * Ko * R', R);
lmiterm([-1, 3, 3, 0], -(R + R')/2);

%for P,J,E>0

```

```

lmiterm([-2, 1, 1, P], 1, 1);
lmiterm([-3, 1, 1, J], 1, 1);
lmiterm([-4, 1, 1, E], 1, 1);

% Add a small margin to help the solver
lmiterm([-2, 1, 1, 0], -1e-6 * eye(2));
lmiterm([-3, 1, 1, 0], -1e-6 * eye(2));
lmiterm([-4, 1, 1, 0], -1e-6 * eye(2));
lmisys = getlmis;

options = [0, 0, 0, 0, 0]; % Default solver options
[tmin, xfeas] = feasp(lmisys, options);

```

Solver for LMI feasibility problems $L(x) < R(x)$
This solver minimizes t subject to $L(x) < R(x) + t \cdot I$
The best value of t should be negative for feasibility

Iteration : Best value of t so far

```

1          0.516546
2          0.465195
***          new lower bound:      0.241222

```

Result: best value of t : 0.465195
f-radius saturation: 0.000% of $R = 1.00e+09$

These LMI constraints were found infeasible

```

% Check if a solution exists
if tmin < 0
    disp('Success! The system is stable and passive.');
```

 % Extract the actual matrices

```

    P_sol = dec2mat(lmisys, xfeas, P);
    J_sol = dec2mat(lmisys, xfeas, J);
    E_sol = dec2mat(lmisys, xfeas, E);

    disp('Matrix P:'); disp(P_sol);
    disp('Matrix J:'); disp(J_sol);
    disp('Matrix E:'); disp(E_sol);
else
    disp('Infeasible. No stable solution found for these parameters.');
```

end

Infeasible. No stable solution found for these parameters.

```

% --- SIMULATION FOR THE GRAPH ---
k_max = 100;
x = zeros(2, k_max);
x(:,1) = [0.2; -0.2]; % Initial Condition

for k = 1:k_max-1
    % 1. Define w(k) based on the paper

```

```

if k <= 50
    wk = [cos(k); sin(1.2*k)];
else
    wk = [0; 0];
end

% 2. Calculate the output y(k)
yk = A*x(:,k) + R*wk;

% 3. Apply a Nonlinearity (Saturation example for f(y))
% We use 'sat' to simulate the [Ko, Kq] = [-1, 1] hardware limit
fk = max(min(yk, 1), -1);

% 4. Update state x(k+1)
x(:,k+1) = fk + S*wk;
end

% --- PLOTTING ---
figure;
plot(0:k_max-1, x(1,:), '-r', 'LineWidth', 1.5); hold on;
plot(0:k_max-1, x(2,:), '--b', 'LineWidth', 1.5);
grid on;
xlabel('Time step (k)');
ylabel('States x(k)');
legend('x1(k)', 'x2(k)');
title('State Trajectories with Disturbance Nullified at k=50');

```

